Hochschule Fulda
University of Applied Sciences

Global Distributed Software Development - AI5088 (WiSe25/26)

Project Title: FuldaNexus

Team Number:

Group 02 Team

Members

No:	Name	Email	Role
1	Peddada Ramachandra Sai Darshan	ramachandra-sai-darshan.peddada@informatik.hs-fulda.de	Lead + QA
2	Sahaj Bikram Malla	Sahaj bikram.malla@informatik.hs-fulda.de	Backend Lead
3	Tanmay Manjunath	tanmay.manjunath@informatik.hs-fulda.de	Frontend Lead
4	Meet Kalathiya	meet-mukeshbhai.kalathiya@informatik.hs-fulda.de	Frontend
5	Karan Kamath	karan-karunakar.kamath@informatik.hs-fulda.de	Backend
6	Bikrant Bidari	bikrant.bidari@informatik.hs-fulda.de	Git Master

Class CEO/CTO

Prof. Rainer
Todtenhoefer Fulda
University, Germany

rainer.todtenhoefer@informatik.hs-fulda.de

1. Product Summary

Fulda Nexus is a visionary, next-generation event management platform designed specifically for the **University of Applied Sciences Fulda**. Developed by a dedicated team of six, it transforms the campus experience into a seamless, engaging, and safe journey for both students and organizers. The platform serves as a digital community cornerstone where students can discover everything from academic workshops to cultural festivals through an intuitive, web-responsive interface.

What sets **Fulda Nexus** apart are its specialized features tailored to the unique dynamics of university life:

- **Event Safety SOS Co-pilot**: This essential security feature provides a panic button within the event chat interface that immediately alerts admin with the user's real-time location and event context.
- **Event-Specific Community Chat**: A dedicated chat room is automatically generated for every approved event, accessible exclusively to registered attendees. This fosters deep group cohesion, allowing students to network, coordinate, and exchange ideas from preparation through follow-up.
- **Secure Ticket Reselling**: To accommodate changing student schedules, the platform includes a secure in-app marketplace. It facilitates safe ownership transfers at or below the original price, ensuring events remain at capacity while following university rules.
- **Gamified Reward Engine**: Users earn points and badges for attendance and platform interactions. These achievements are visible on user profiles and can be redeemed for rewards such as discounts and coupons.

Built with a robust, modern technology stack featuring **React 19** for the frontend, **FastAPI (Python)** for the backend, and **AWS Cloud Infrastructure**, Fulda Nexus is engineered to provide a high-performance, scalable solution for the modern campus.

1 Functional Requirement

1.1 Common Features

- **Register and User Authentication-(P1)** The system allows university users to register with a verified campus email and log in securely. Guest access is allowed for limited browsing.
- **User Roles-(P1)** The system shall support distinct roles (Attendee, Event Organizer, Administrator) with specific permissions.
- **Profile Management-(P1)** Users shall be able to view and edit profile details (name, contact, interests).
- **Password Management-(P1)** Users shall be able to reset their passwords.

1.2 Event Creation & Management (Organizer)

- **Create Events-(P1)** Authorized Organizers shall be able to create event listings with title, description, date, time, location (physical/virtual), capacity, and category.
- **Ticket Management-(P1)** Organizers shall specify ticket types (free, paid, VIP), prices, quantities, and availability.
- **Media Upload-(P1)** Organizers shall be able to upload event-related images and videos.
- **Attendee Management-(P1)** Organizers shall manage attendee lists and check in attendees.
- **Edit/Cancel-(P2)** Organizers shall be able to view, edit, or cancel their own events.

1.3 Event Discovery & Search

- **Event Listing-(P1)** The system shall display a comprehensive list of upcoming and past events.
- **Basic Search-(P1)** Users shall be able to search by keywords, categories, dates, and locations.
- **Event Details-(P1)** The system shall display details including schedule, location map, and organizer contact.
- **Advanced Filtering-(P2)** Users shall be able to filter by "free events," departments, or event type.
- **Semantic Search -(P4)** The system shall support natural language queries (e.g. "workshops for public speaking") and provide recommendations based on semantic understanding and user interests.
- **Favourites-(P4)** Users shall be able to add events to a personal "watch" list.

1.1 Registration & Ticketing

- **Registration-(P1)** Users shall be able to register for free events or purchase tickets for paid events.
- **Ticket Generation-(P1)** The system shall generate unique digital tickets/QR codes for successful registrations.
- **My Tickets-(P1)** Users shall view a record of their registered/purchased tickets in their profile.

1.4 Event Safety SOS Co-pilot

- **SOS Button-(P1)** The system shall provide an easily accessible panic button within the event Chat interface.
- **Security Alert-(P1)** Activation shall immediately alert Event organizer with the user's ticket details and users live location.
- **Incident Logging-(P1)** The incident is logged in the SOS Dashboard for the for the admins.
- **Incident Verification-(P1**) Admin shall proceed with the further action or he may reject the peal.

1.5 Ticket Reselling

- **List for Resale-(P1)** Users shall be able to list purchased tickets for resale.
- **Secure Transfer-(P1)** The system shall facilitate the secure transfer of ticket ownership from seller to buyer.
- **Resale Marketplace-(P1)** A dedicated resale market place for browsing resale tickets.

1.6 Community Chat

- **Event Chat Rooms-(P1)** A dedicated chat room shall be created for each event, accessible only to registered attendees and event organizer.
- **Messaging-(P1)** Users shall be able to send text messages within the chat.
- **Moderation-(P1)** The system shall support reporting of inappropriate content in chats.

1.7 Rewards

- **Track Engagement-(P1)** The system shall track actions like attendance and interactions.
- **Points & Badges-(P2)** Users shall earn points and badges based on predefined criteria.
- **Profile Display-(P2)** Earned badges and points shall be visible on the user profile.

- **Leaderboard-(P2)** A leaderboard shall display top-ranking users.
- **Reward Redemption-(P1)** Users shall be able to redeem points for rewards (e.g. discounts. Coupons).

URL – FuldaNexus

2 Usability test Plan

Function to be Tested: Emergency SOS feature (by ticket purchased user)

Test Objective

The purpose of this usability test is to assess the effectiveness, efficiency, and user satisfaction of the SOS and Resale Functionality within the FuldaNexus web application. Since this is a critical safety feature, the response time and ease of use are paramount. The test aims to:

- **Evaluate Accessibility:** Determine if the SOS button is immediately visible and accessible within the Event Chat interface during a simulated emergency.
- **Verify Data Accuracy:** Ensure that the system correctly captures and transmits the user's live location and ticket details to the administrator.
- **Assess User Feedback:** Confirm that the user receives clear, immediate feedback that help is on the way.
- **Validate Admin Workflow:** Verify that the administrator (Organizer/Security) receives the alert on the SOS dashboard and can successfully close the issue once addressed.
- **Ease of Listing:** Evaluate how easily a user can locate the "Resell" option in "My Tickets" and set a valid price.
- **Policy Compliance:** Test if the system clearly communicates eligibility (eg, preventing resale if disabled) and price caps.
- **Marketplace Discovery:** Assess how easily a buyer can distinguish resale tickets from standard tickets in the marketplace.

Test Background and Setup

- **System Setup:** The test will be conducted on the FuldaNexus web application. The environment requires two active sessions:
    1. **Student View:** Logged in as a registered student (e.g., Persona: Lena or Sara) who has purchased a ticket for an active event.
    2. **Admin View:** Logged in as an Event Organizer or Administrator (e.g., Persona: Dr. Markus Weber) with access to the SOS Dashboard.
- **Starting Point:**
    - The student tester will begin on the Home page or directly inside the specific "Event Chat Room".
    - The admin tester will be on the "Organizer Dashboard" monitoring active events.
- **Intended Users:** Students and Staff of Fulda University of Applied Sciences.

Testing Users Detail

To ensure diverse feedback, the following user profiles (based on project Personae) will simulate the testing:

1. **Lena (The Event Enthusiast):** Tech-savvy bachelor student familiar with the app.
2. **Sara (International Student):** Safety-conscious user, testing the SOS reliability.
3. **Dr.Markus Weber (Organizer):** Testing the reception of SOS alerts.
4. **Emma (Club Rep):** Testing the interface for resale permissions.

5. **Novice User:** A student with low tech skills to ensure the SOS button is intuitive.

What is to Be Measured

The test will measure user satisfaction and confidence in the safety feature using a Likert scale questionnaire (1-5). Key metrics include:

- **User Satisfaction:** The primary purpose is to measure user satisfaction with the specific functionality being tested.
- **Evaluation Method:** This is evaluated through a short questionnaire containing six questions using a Likert scale, which users complete after finishing their testing.
- **Effectiveness:** This is measured based on the users' ability to complete tasks confidently and whether the experience feels intuitive and aligns with their expectations. It assesses if users can understand the feature without guidance and identify relevant results.
- **Efficiency:** This is measured using the Likert scale where users rate their experience regarding ease of use, speed, and overall design. Key indicators include the perceived time taken to complete the search and the smoothness of the process. An average rating is calculated to determine if the feature is efficient or needs improvement

Usability Task Description
Instructions for the Tester (Student Role):

1. **Login**: Log in to FuldaNexus using the provided student credentials.
2. **Navigate to Chat:** Go to "My Tickets" or "Upcoming Events," select the active event "Fulda Night Fest," and enter the Community Chat Room.
3. **Simulate Emergency:** Imagine a scenario where you feel unsafe or have a medical issue.
4. **Activate SOS:** Locate the SOS / Panic Button within the chat interface and click it.
5. **Confirm Action:** If prompted, confirm the sharing of your live location (or select your zone if GPS is unavailable).
6. **Observe Feedback:** Note if the system provides a confirmation message (eg, "Security Alerted").

Instructions for the Tester (Admin Role):

1. **Monitor Dashboard:** Watch the Organizer Dashboard for incoming alerts.
2. **Verify Alert:** Confirm that a real-time notification appears with the student's name, ticket details, and location.
3. **Resolve:** Mark the incident as "Addressed/Resolved" and close the issue.

Measuring Effectiveness
The feature is considered effective if:

- The student can locate and trigger the SOS button within 5 seconds of entering the chat.
- The admin receives the notification within 2 seconds of activation (Real-time requirement).
- The location data displayed to the admin accurately reflects the student's position or selected zone.

Measuring Efficiency
Efficiency is measured via the post-test questionnaire. High ratings indicate that the security mechanism is unobtrusive yet easily accessible when needed, and the workflow for the admin is streamlined.

User Satisfaction Questionnaire

Please indicate how much you agree with the following statements based on your experience using the "SOS" feature of FuldaNexus:

Scale:

- Strongly Disagree 😠
- Disagree😐
- Neutral 😶
- Agree 🙂
- Strongly Agree 😃

code	Statement	feature
S1	The SOS button was easy to find within the chat room.	SOS
S2	I felt confident that the Admin received my location/details.	SOS
S3	The steps to resell my ticket were clear and simple.	Resale
S4	It was clear which tickets were valid for resale and which were not.	Resale
S5	The price limit rule (anti-scalping) was clearly communicated.	Resale
S6	I am satisfied with the overall safety and marketplace features.	Overall

Tester Responses

Usability Features	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree	Average Score
S1 (SOS Visibility)	**0**	**0**	**0**	**1**	**4**	**4.8**
S2 (SOS Feedback)	**0**	**1**	**0**	**2**	**2**	**4.0**
S3 (Resale Ease)	**0**	**0**	**0**	**2**	**3**	**4.6**
S4 (Resale Clarity)	**0**	**0**	**1**	**3**	**1**	**4.0**

Usability Features	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree	Average Score
S5 (Price Rules)	**0**	**1**	**1**	**1**	**2**	**3.8**
S6 (Overall)	**0**	**0**	**0**	**2**	**3**	**4.6**

Inference

The usability test results indicate that the **SOS Co-pilot** is highly accessible, with users easily locating the button inside the chat (Score 4.8). However, the feedback mechanism (S2) scored lower (4.0), suggesting that while the alert is sent, users may need stronger visual confirmation (eg, a "Help is on the way" banner) to feel truly secure.

For **Resale**, the process is efficient (S3), but the communication regarding price caps (S5) caused some friction. Users attempted to enter higher prices before realizing the limit. The UI should proactively display the maximum allowable price (Original MRP) *before* the user attempts to input a value to improve this interaction. Overall, the features are functional and align with the persona goals of **Sara** (Safety) and **Lena** (Event Participation).

3 QA Test Plan for FuldaNexus

3.1 Test Objectives

To validate the reliability, security, and functionality of the core security and marketplace features. This includes:

- **SOS Co-pilot:** Ensuring immediate accessibility of the panic button, accurate location/zone transmission, and real-time alert delivery to organizers/security.
- **Ticket Resale:** Verifying the secure transfer of ownership, enforcement of price caps (anti-scalping), and adherence to organizer permissions.
- **System Integrity:** Confirming that alerts and transactions are logged correctly for administrative audit.

3.2 Hardware and Software Setup

- **Hardware:** Desktop/Laptop (Admin/Organizer view), Mobile Device (Student/Attendee view).
- **Software:**
    - **Frontend:** React 19 + Tailwind CSS.
    - **Backend:** FastAPI (Python).
    - **Browsers:** Google Chrome (Latest), Mozilla Firefox (Latest).
    - **Database:** MySQL 8.0 on AWS RDS.

3.3 Feature 1: Event Safety SOS Co-pilot

Description: A safety feature allowing attendees to trigger an emergency alert from the event chat interface, sending location details to the organizer.

Test #	Test Title	Test Description	Test Input	Expected Output	status
1	**SOS Button Visibility**	Verify that the SOS button is easily accessible within the event interface for a registered attendee.	Login as **Lena** (student). Join “Tech Workshop” Chat Room.	"SOS" or Panic button is clearly visible in the chat interface.	Pass
2	**Trigger Alert**	Verify that triggering SOS sends specific location details to Admin	Click SOS button. And sent message has been shown	Alert sent immediately. Organizer receives notification with live location	Pass
4	**Admin Incident Receipt**	Verify that the Admin receives the alert in real-time on the dashboard.	Login as **Dr. Weber** (Admin). Monitor Dashboard while User triggers SOS.	Real-time notification appears with User Name, Location.	Pass

Test #	Test Title	Test Description	Test Input	Expected Output	status
5	**Incident Logging**	Ensure the incident is recorded in the system logs for auditing.	Login as **Jonas** (Admin). Check system/security logs.	A new entry exists in AuditLogor Reporttable with TargetType: EventandReason: SOS.	Pass
6	**Incident Resolution**	Verify that the admin can close the alert after addressing it.	As **Admin**, select the active SOS incident and click "Resolve".	Incident status updates to "Resolved". Communication channel closes.	Pass

3.4 Feature 2: Secure Ticket Reselling

Description: A secure in-app marketplace allowing users to resell tickets they cannot use, with price caps and ownership transfer.

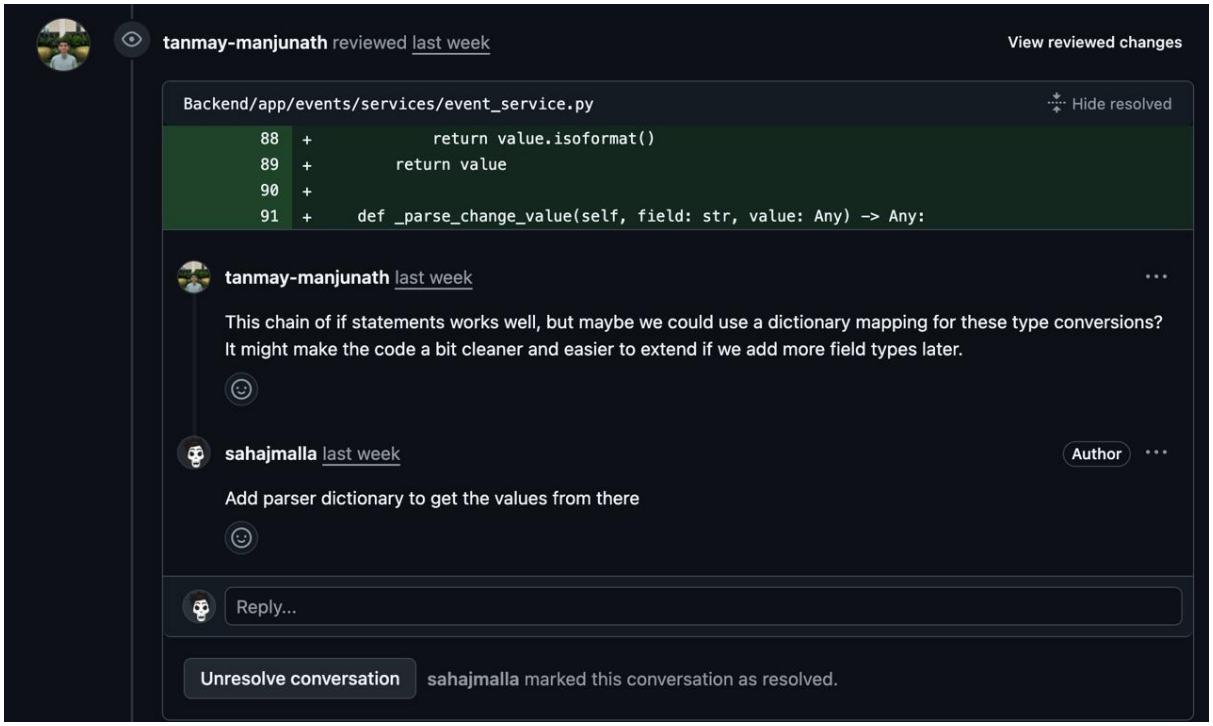
Test #	Test Title	Test Description	Test Input	Expected Output	status
1	**List Ticket for Resale**	Verify a user can list a valid ticket for resale.	Log in as **Sara** . Go to "My Tickets". Select "Movie Night". Click "Resell". Enter Price: €10 (Original: €10).	Ticket is listed on the internal marketplace. Status changes to "ResellEligible" or "Listed".	Pass
3	**Resale Eligibility Check**	Verify users cannot resell tickets for non-refundable/blocked events.	Select a ticket for an event where Organizer disabled reselling.	"Resell" option is disabled or shows error "Reselling not permitted by organizer".	Pass
4	**Marketplace Visibility**	Verify other users can see the listed ticket.	Login as **Lena** . Go to "Resale Marketplace" or Event Page.	Sara's ticket is visible and available for purchase.	Pass
5	**Secure Ownership Transfer**	Verify ownership transfers upon purchase.	**Lena** buys **Sara's** ticket.	Transaction successful. Sara's ticket removed/invalidated.Lena sees new QR code in "My Tickets".	Pass

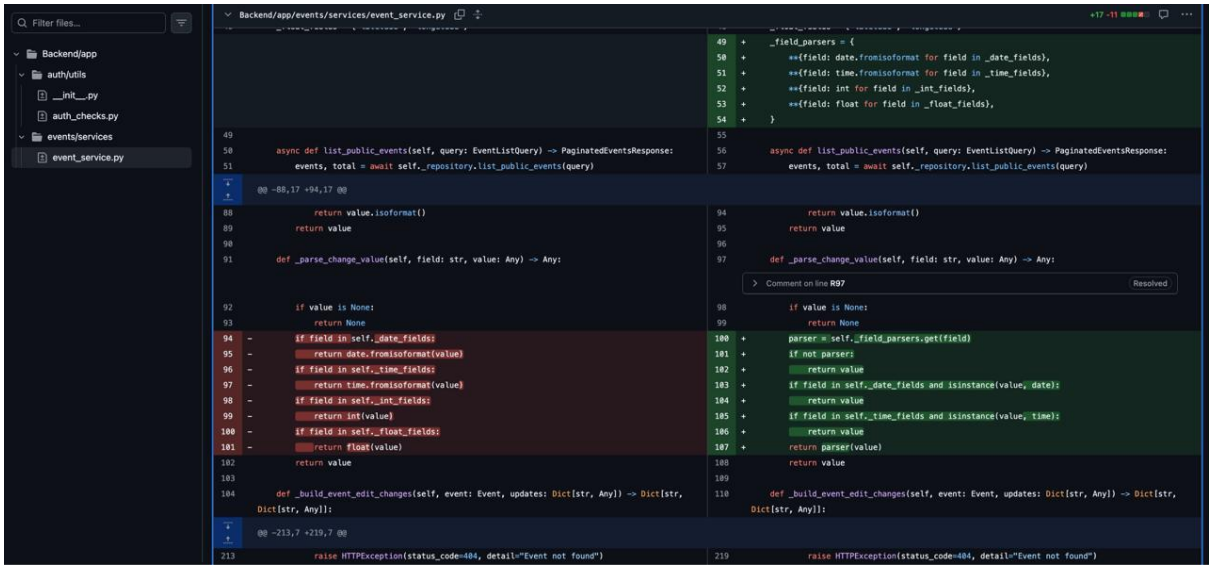
4 Code Review

Code Review #1

Submitted by: Sahaj

Reviewed by: Tanmay





tanmay-manjunath reviewed last week

View reviewed changes

frontend/src/components/userDashboardTabs/EventEditRequests.jsx

Hide resolved

```
70 +       .finally(() => setLoading(false));
71 +   }, [user, page, pageSize]);
72 +
73 +   const getStatusBadge = (status) => {
```

tanmay-manjunath last week

The switch statement is clear, but you could also do this with a simple object lookup (e.g. const statusColors = { approved: '...', ... }). It might save a few lines of boilerplate code.

sahajmalla last week

Author

Added object lookup

Reply...

Unresolve conversation

sahajmalla marked this conversation as resolved.

Merged

Add edit events backend and frontend #94

Commit 44be141

tanmay-manjunath merged 3 commits into main from edit-events

0 / 2 viewed

Comments 0

Submit comments

Filter files...

- frontend/src/components
  - adminDashboardTabs
    - EventModification.jsx
  - userDashboardTabs
    - EventEditRequests.jsx

frontend/src/components/userDashboardTabs/EventEditRequests.jsx

+9 -18

Viewed

```
29      const { user } = useSelector((state) => state.auth);
30      const [requests, setRequests] = useState([]);
@@ -70,16 +68,15 @@
70          .finally(() => setLoading(false));
71      }, [user, page, pageSize]);
72
73      const getStatusBadge = (status) => {
74 -    switch (status?.toLowerCase()) {
75 -      case "approved":
76 -        return "text-green-600 border-green-200 bg-green-50";
77 -      case "rejected":
78 -        return "text-red-600 border-red-200 bg-red-50";
79 -      case "pending":
80 -      default:
81 -        return "text-yellow-600 border-yellow-200 bg-yellow-50";
82 -    }
83      };
84
85      const formatValue = (field, value) => {
@@ -96,11 +93,6 @@
96        return String(value);
97      };
98
99 -    const isLongDescription = (value) => {
```

```
27      const { user } = useSelector((state) => state.auth);
28      const [requests, setRequests] = useState([]);
68          .finally(() => setLoading(false));
69      }, [user, page, pageSize]);
70
71 +    const statusColors = {
72 +      approved: "text-green-600 border-green-200 bg-green-50",
73 +      rejected: "text-red-600 border-red-200 bg-red-50",
74 +      pending: "text-yellow-600 border-yellow-200 bg-yellow-50",
75 +    };
76 +
77      const getStatusBadge = (status) => {
```

Comment on line R77 Resolved

```
78 +      const key = status?.toLowerCase() || "pending";
79 +      return statusColors[key] || statusColors.pending;
80      };
81
82      const formatValue = (field, value) => {
93        return String(value);
94      };
95
```

Merged **Add edit events backend and frontend** #94

Commit 44be141 · tanmay-manjunath merged 3 commits into main from edit-events

0 / 2 viewed · Comments 0 · Submit comments

Filter files...

- frontend/src/components
  - adminDashboardTabs
    - EventModification.jsx
  - userDashboardTabs
    - EventEditRequests.jsx

frontend/src/components/userDashboardTabs/EventEditRequests.jsx +9 -18 Viewed

```diff
29      const { user } = useSelector((state) => state.auth);
30      const [requests, setRequests] = useState([]);
@@ -70,16 +68,15 @@
70          .finally(() => setLoading(false));
71      }, [user, page, pageSize]);
72
73      const getStatusBadge = (status) => {
74  -     switch (status?.toLowerCase()) {
75  -       case "approved":
76  -         return "text-green-600 border-green-200 bg-green-50";
77  -       case "rejected":
78  -         return "text-red-600 border-red-200 bg-red-50";
79  -       case "pending":
80  -       default:
81  -         return "text-yellow-600 border-yellow-200 bg-yellow-50";
82  -     }
83      };
84
85      const formatValue = (field, value) => {
@@ -96,11 +93,6 @@
96        return String(value);
97      };
98
99  - const isLongDescription = (value) => {
```

```diff
27      const { user } = useSelector((state) => state.auth);
28      const [requests, setRequests] = useState([]);
68          .finally(() => setLoading(false));
69      }, [user, page, pageSize]);
70
71  +   const statusColors = {
72  +     approved: "text-green-600 border-green-200 bg-green-50",
73  +     rejected: "text-red-600 border-red-200 bg-red-50",
74  +     pending: "text-yellow-600 border-yellow-200 bg-yellow-50",
75  +   };
76  +
77      const getStatusBadge = (status) => {
78  +     const key = status?.toLowerCase() || "pending";
79  +     return statusColors[key] || statusColors.pending;
80      };
81
82      const formatValue = (field, value) => {
93        return String(value);
94      };
95
```

Comment on line R77 · Resolved

ckend and frontend #94

tanmay-manjunath merged 3 commits into main from edit-events

Comments 0 · Submit comments

Backend/app/auth/utils/auth_checks.py +6

```diff
58
59      def is_standard_user(user: User) -> bool:
60          return has_role(user, "user")
```

```diff
58
59      def is_standard_user(user: User) -> bool:
60          return has_role(user, "user")
61  +
62  +
63  + def is_admin_or_owner(user: User, owner_id: Optional[str]) -> bool:
64  +     if not owner_id:
65  +         return False
66  +     return is_admin(user) or user.id == owner_id
```

ckend and frontend #94

tanmay-manjunath merged 3 commits into main from edit-events

Comments 0 · Submit comments

Backend/app/auth/utils/auth_checks.py +6

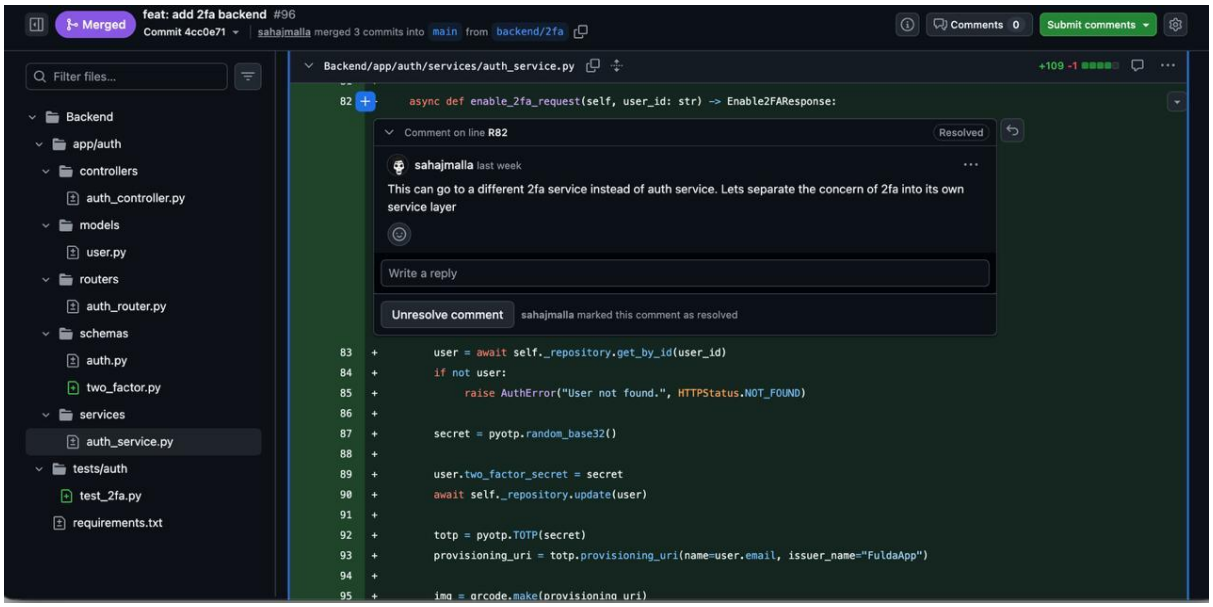
58
59 def is_standard_user(user: User) -> bool:
60 return has_role(user, "user")

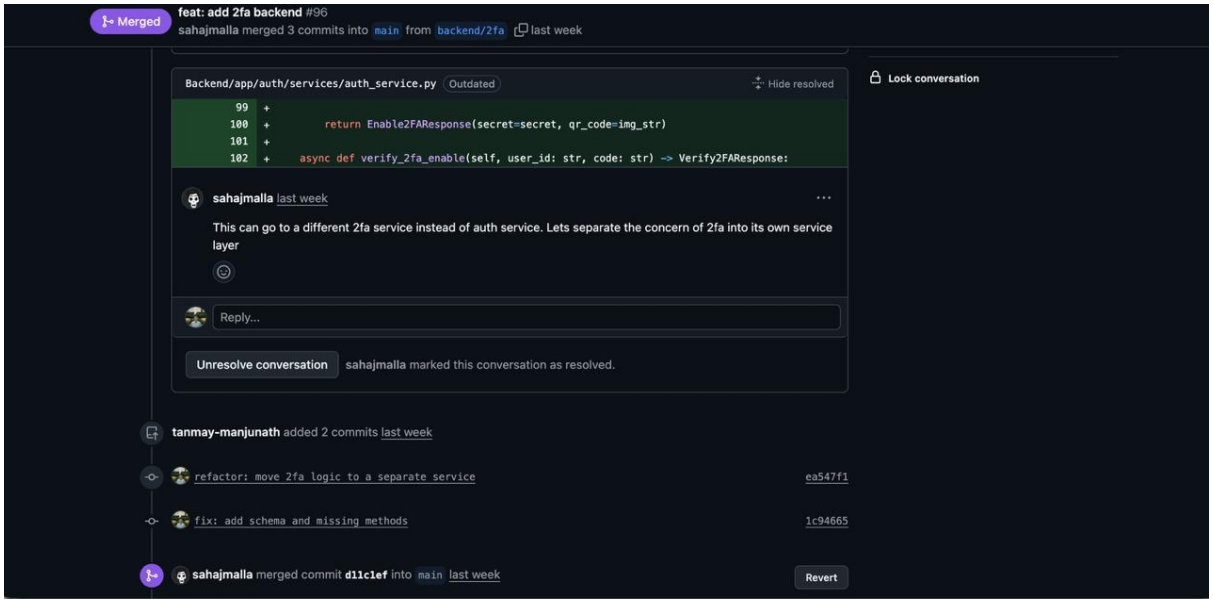
58
59 def is_standard_user(user: User) -> bool:
60 return has_role(user, "user")
61 +
62 +
63 + def is_admin_or_owner(user: User, owner_id: Optional[str]) -> bool:
64 + if not owner_id:
65 + return False
66 + return is_admin(user) or user.id == owner_id

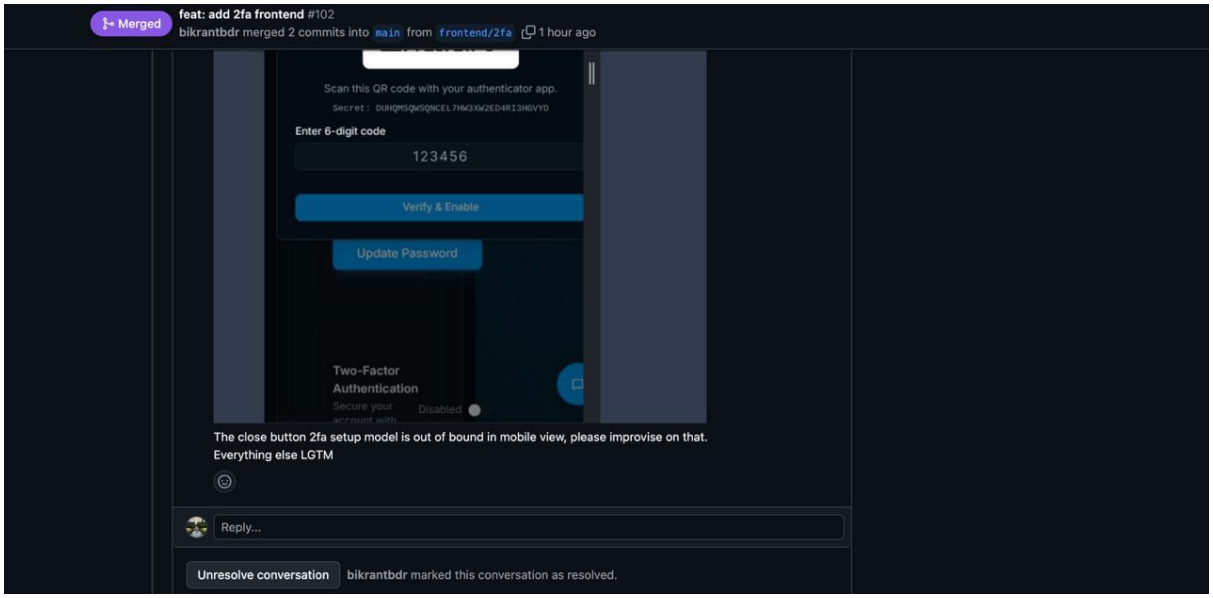
Code Review #2

Submitted by: Tanmay

Reviewed by: Bikrant, Sahaj





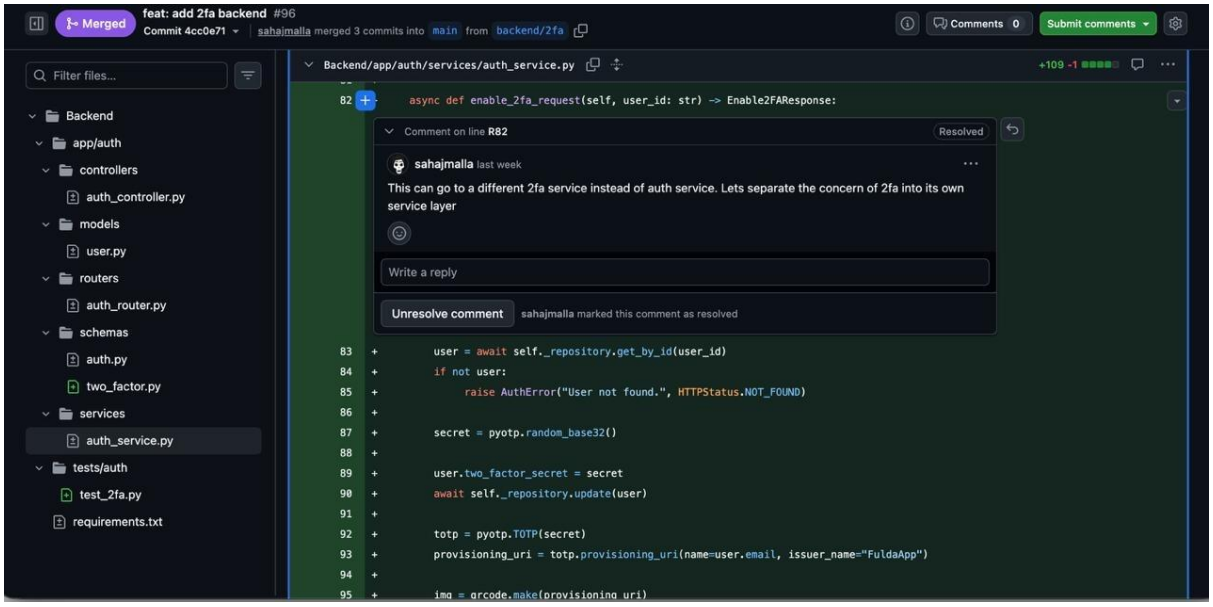


Code Review #2

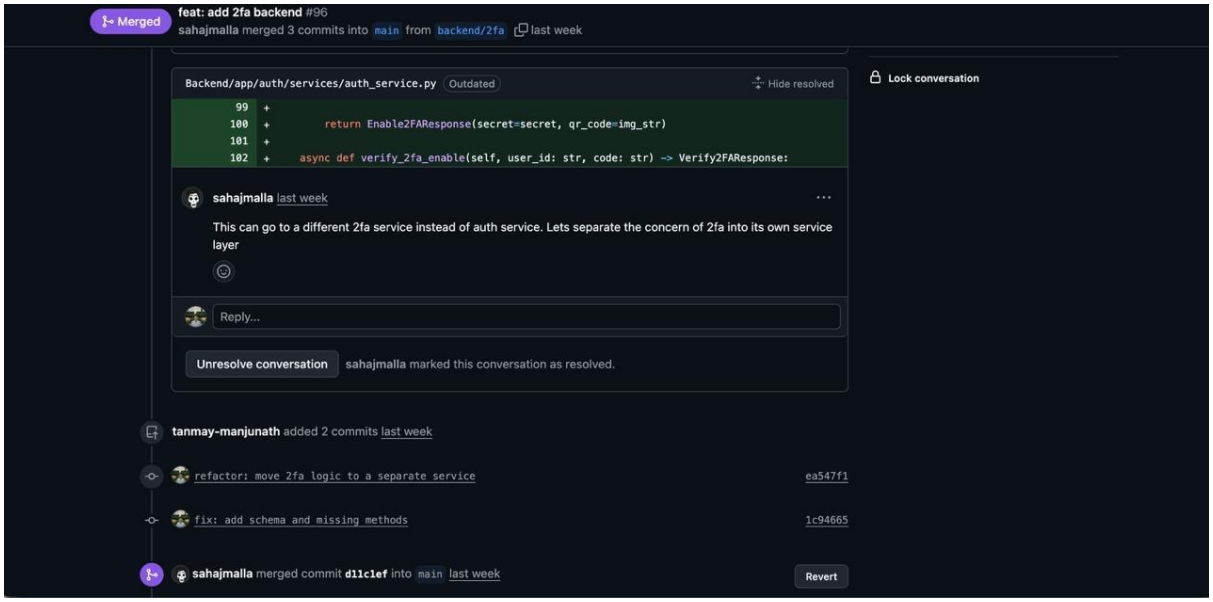
Submitted by: Tanmay

Reviewed by: Bikrant

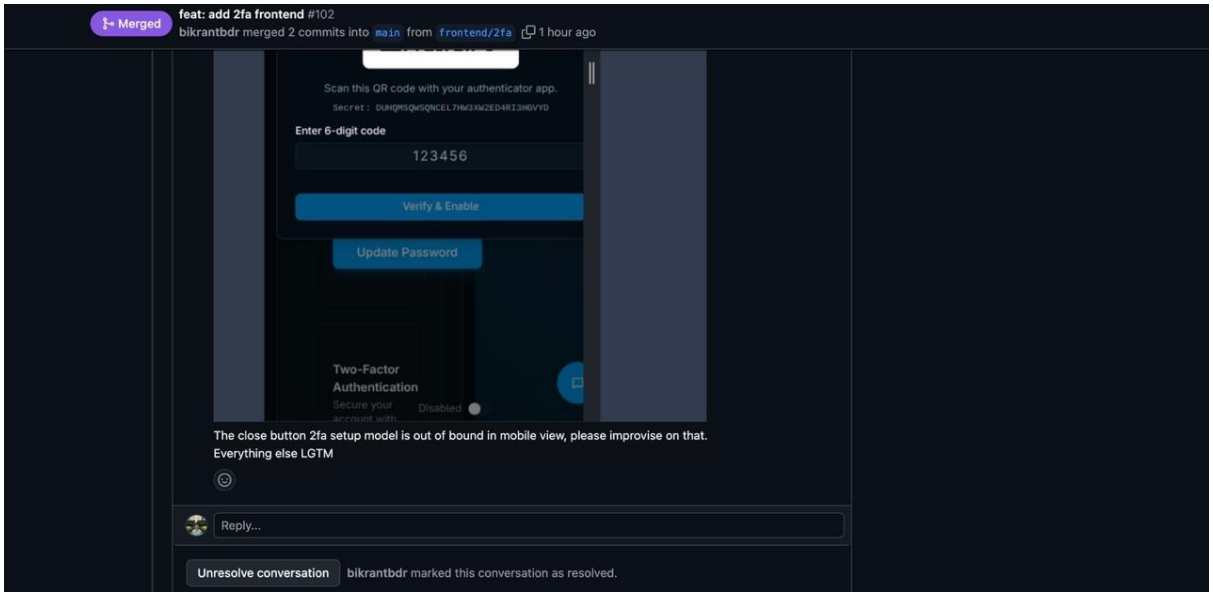
Review Comment:



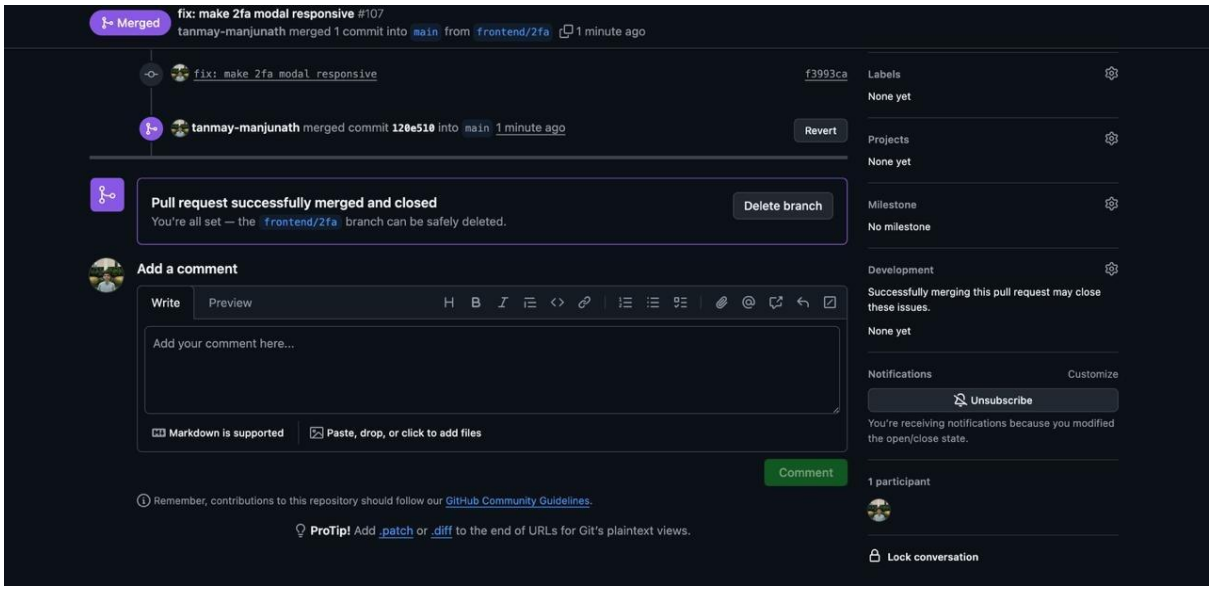
After Changes:



Review Comment:



After Changes:



Code Review #4

Feature: Rewards

Submitted by: Karan Kamath

Reviewed by: Meet Kalathiya

Suggestion #1

Review Comment :

```
frontend/src/services/rewardApiService.js
+   /**
+    * Get current user's streak information
+    */
+   getMyStreak: async () => {
```

github-meetk 5 hours ago

Use try-catch block when calling API, just to handle unexpected errors.

After Changes :

```
frontend/src/services/rewardApiService.js                +26 -8   Viewed
+       );
+     }
    },

    /**
     * Get all streak multiplier tiers
     */
    getStreakMultipliers: async () => {
-     const response = await axios.get(`${baseUrl}rewards/streak/multipliers`, {
-       headers: getAuthHeaders(),
-     });
-     return response.data;
+     try {
+       const response = await axios.get(`${baseUrl}rewards/streak/multipliers`, {
+         headers: getAuthHeaders(),
+       });
+       return response.data;
+     } catch (error) {
+       console.error("Error fetching streak multipliers:", error);
+       throw new Error(
+         error.response?.data?.detail ||
+         error.message ||
+         "Failed to fetch streak multipliers"
+       );
+     }
    }
```

Suggestion #2

Review Comment :

github-meetk requested changes 5 hours ago

View reviewed changes

frontend/src/components/rewards/StreakDisplay.jsx

```
+     weeks_to_next_milestone = null,
+   } = streak;
+
+   // Calculate progress to next milestone
```

github-meetk 5 hours ago

This calculation runs on every render. Instead you can use useMemo because it depends on current_streak and next_milestone values.

After Changes :

...src/components/rewards/StreakDisplay.jsx +7 -5 Viewed

```
@@ -1,4 +1,4 @@
- import React from "react";
+ import React, { useMemo } from "react";
  import {
    Flame,
    Target,

@@ -56,10 +56,12 @@ const StreakDisplay = ({ streak, loading = false }) => {
      weeks_to_next_milestone = null,
    } = streak;

-   // Calculate progress to next milestone
-   const progressToMilestone = next_milestone
-     ? (current_streak / next_milestone.weeks) * 100
-     : 100;
+   // Calculate progress to next milestone (memoized to avoid recalculation on every render)
+   const progressToMilestone = useMemo(() => {
+     return next_milestone
+       ? (current_streak / next_milestone.weeks) * 100
+       : 100;
+   }, [current_streak, next_milestone]);
```

Suggestion #3

Review Comment :

frontend/src/store/slices/rewardSlice.js

```
    redemptionError: null,
    redemptionRate: null,
+
+   // Streak
```

github-meetk 5 hours ago

group all the streak related states into a nested object for better organization.

After Changes :

```
frontend/src/store/slices/rewardSlice.js                    +14 -12   Viewed

38      -   streak: null,
39      -   streakLoading: false,
40      -   streakError: null,
41      -   streakMultipliers: [],
    37  +   // Streak - group all streak related states into a nested object for better organization
    38  +   streak: {
    39  +     data: null,
    40  +     loading: false,
    41  +     error: null,
    42  +     multipliers: [],
    43  +   },
42  44    };
43  45
44  46    // Async thunks

      @@ -335,21 +337,21 @@ const rewardSlice = createSlice({

335 337 +
336 338         // Streak
337 339         .addCase(fetchMyStreak.pending, (state) => {
338     -         state.streakLoading = true;
339     -         state.streakError = null;
    340 +         state.streak.loading = true;
    341 +         state.streak.error = null;
340 342         })
341 343         .addCase(fetchMyStreak.fulfilled, (state, action) => {
342     -         state.streakLoading = false;
343     -         state.streak = action.payload;
    344 +         state.streak.loading = false;
    345 +         state.streak.data = action.payload;
```

Code Review #5

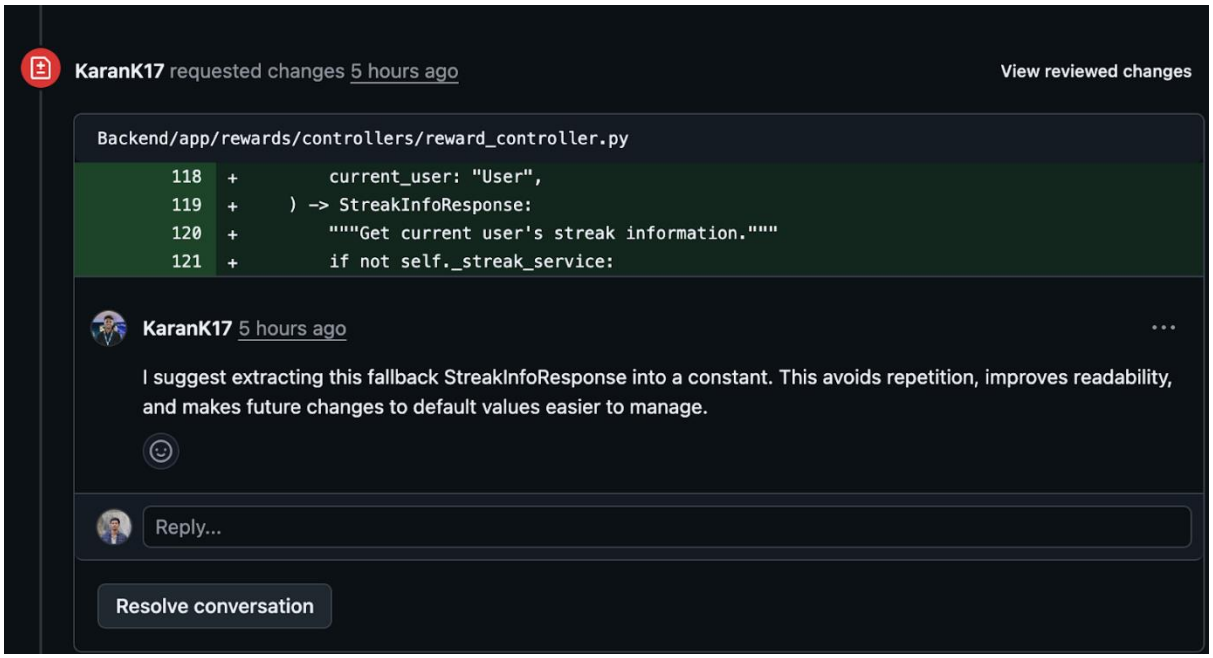
Feature: Rewards

Submitted by: Meet Kalathiya

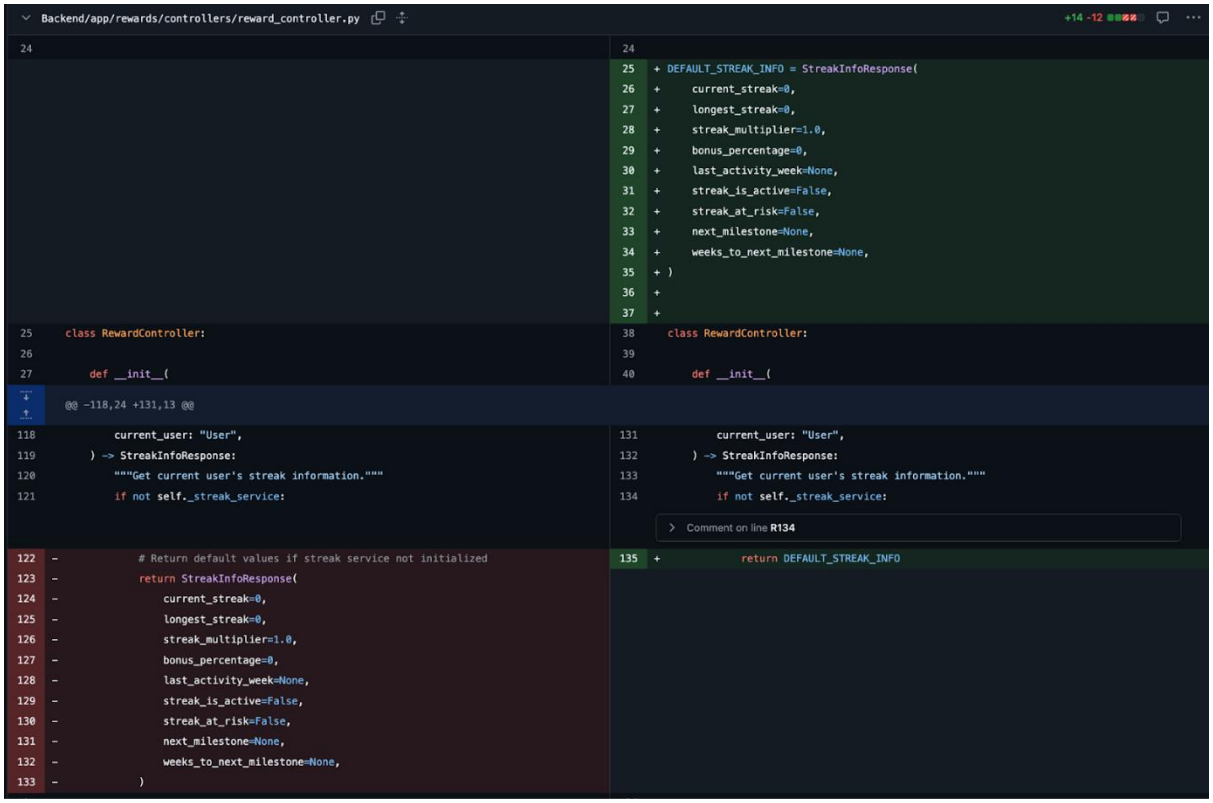
Reviewed by: Karan Kamath

Suggestion #1

Review Comment :



After Changes :



Suggestion #2

Review Comment :

Backend/app/rewards/controllers/reward_controller.py

```
135 +         streak_info = await self._streak_service.get_user_streak_info(current_user.id)
136 +         return StreakInfoResponse(**streak_info)
137 +
138 +     async def get_streak_multipliers(self) -> dict:
```

KaranK17 5 hours ago

Suggestion: Since max_multiplier and max_bonus_percentage are derived from the same multipliers data, it might be safer to compute them once and reuse the value. This avoids repeating calculations and makes the logic slightly more efficient and readable.

Reply...

After Changes :

Backend/app/rewards/controllers/reward_controller.py +4 -2

```
@@ -131,13 +131,13 @@
131          current_user: "User",
132      ) -> StreakInfoResponse:
133          """Get current user's streak information."""
134          if not self._streak_service:
135              return DEFAULT_STREAK_INFO
136
137          streak_info = await self._streak_service.get_user_streak_info(current_user.id)
138          return StreakInfoResponse(**streak_info)
139
140      async def get_streak_multipliers(self) -> dict:
141          """Get all streak multiplier tiers."""
142          from app.rewards.rules.streak_rules import STREAK_CONFIG
143
@@ -153,8 +153,10 @@
153                  "label": f"{weeks}+ weeks" if weeks > 0 else "No streak",
154              })
155
156          return {
157              "tiers": tiers,
158 -            "max_multiplier": max(multipliers.values()),
159 -            "max_bonus_percentage": int((max(multipliers.values()) - 1.0) * 100),
160          }
```

```
131          current_user: "User",
132      ) -> StreakInfoResponse:
133          """Get current user's streak information."""
134          if not self._streak_service:
             Comment on line R134
135              return DEFAULT_STREAK_INFO
136
137          streak_info = await self._streak_service.get_user_streak_info(current_user.id)
138          return StreakInfoResponse(**streak_info)
139
140 +    async def get_streak_multipliers(self) -> dict:
             Comment on line R140
141          """Get all streak multiplier tiers."""
142          from app.rewards.rules.streak_rules import STREAK_CONFIG
143
153                  "label": f"{weeks}+ weeks" if weeks > 0 else "No streak",
154              })
155
156 +        max_multiplier = max(multipliers.values())
157 +
158          return {
159              "tiers": tiers,
160 +            "max_multiplier": max_multiplier,
161 +            "max_bonus_percentage": int((max_multiplier - 1.0) * 100),
162          }
```

Code Review #6

Feature: Backed PDF and QR Generation

Submitted by: Peddada Darshan
Reviewed by: Sahaj

Comment 1

Backend/app/tickets/services/pdf_service.py Outdated

```
+            # We will use a try-except to safely skip if missing.
+            from pathlib import Path
+            base_path = Path(__file__).parent.parent.parent.parent # Backend/
+            logo_path = base_path / "app/assets/FuldaNexusLogo.png"
```

sahajmalla 4 hours ago

Keep both of the paths in utils

darshanpeddadad 3 hours ago Author

Moved the both path to utils

After Changes

```
+ from app.utils import BASE_PATH, LOGO_PATH

  class PDFService:
      @staticmethod
@@ -23,15 +24,10 @@
          # Logo and Branding
          try:
              # Load logo (adjust path as needed depending on where you store it in backend container)
-             # For now assuming it's in app/assets/ or similar.
              # If running in Docker, keep in mind relative paths.
              # We will use a try-except to safely skip if missing.
-             from pathlib import Path
-             base_path = Path(__file__).parent.parent.parent.parent # Backend/
-             logo_path = base_path / "app/assets/FuldaNexusLogo.png"
-
-             if logo_path.exists():
-                 logo = ImageReader(str(logo_path))
+             if LOGO_PATH.exists():
+                 logo = ImageReader(str(LOGO_PATH))
              c.drawImage(logo, 20 * mm, height - 35 * mm, width=40 * mm, height=20 * mm, preserveAspectRatio=True, mask='auto')
```

Comment 2

Backend/app/tickets/services/pdf_service.py

```
+            c.drawString(65 * mm, height - 28 * mm, "FuldaNexus")
+
+        except Exception as e:
+            print(f"Warning: Could not load logo: {e}")
```

sahajmalla 4 hours ago

Remove the print statements as it is not needed.

After Changes

Backend/app/tickets/services/pdf_service.py

```
@@ -35,7 +35,6 @@ def generate_ticket_pdf(event_data: dict, user_data: dict, booking_id: str) -> i
            c.drawString(65 * mm, height - 28 * mm, "FuldaNexus")

        except Exception as e:
-           print(f"Warning: Could not load logo: {e}")
            # Fallback text if logo fails
            c.setFont("Helvetica-Bold", 16)
            c.drawString(20 * mm, height - 30 * mm, "FuldaNexus")
```

Comment 3

Backend/app/tickets/services/ticket_service.py Outdated

```
271 +        stmt = (
272 +            select(Ticket)
273 +            .options(selectinload(Ticket.event), selectinload(Ticket.owner))
274 +            .where(Ticket.id == ticket_id)
```

sahajmalla 4 hours ago

Keep the database actions in repository layer

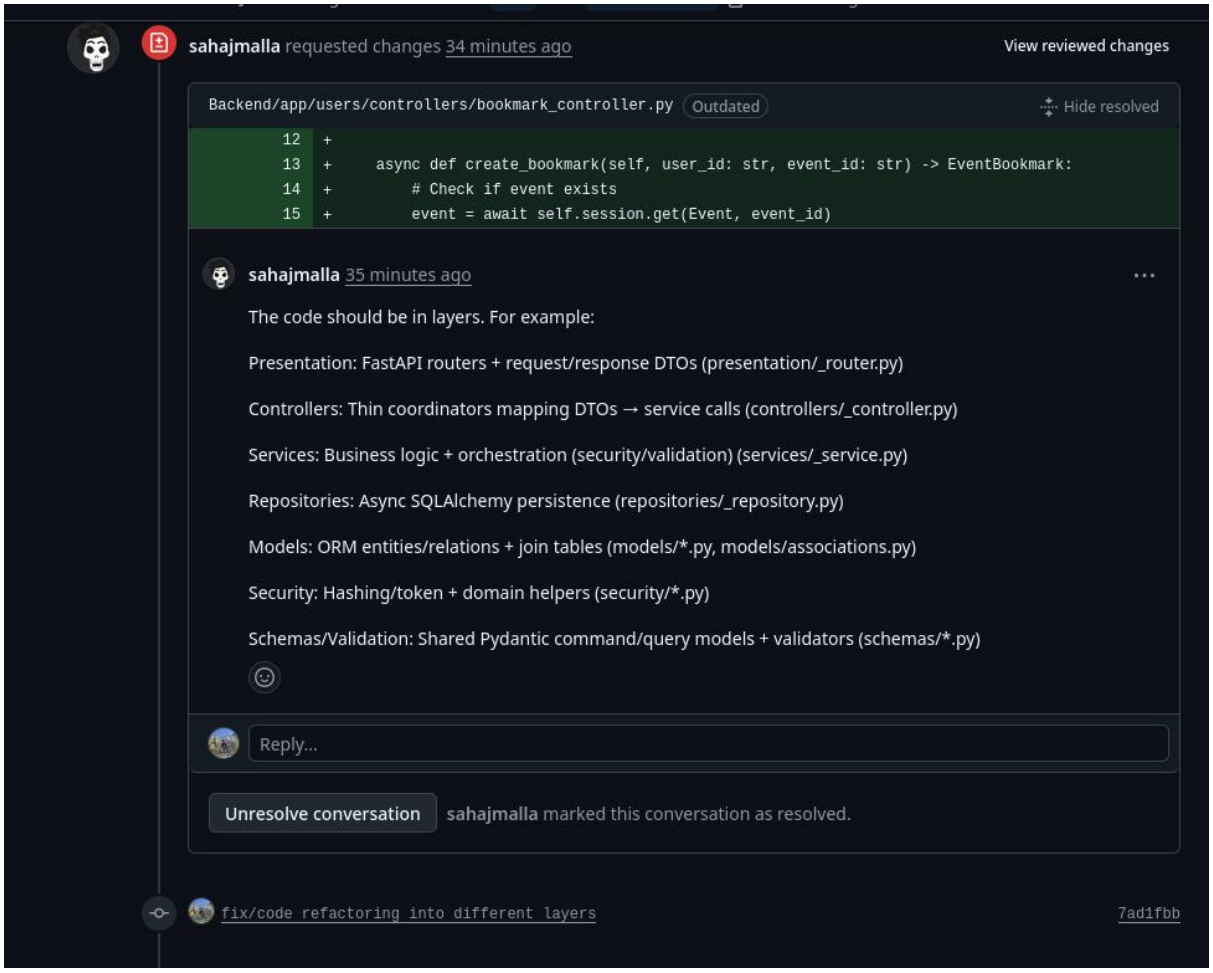
After Changes

Backend/app/tickets/repositories/ticket_repository.py +26

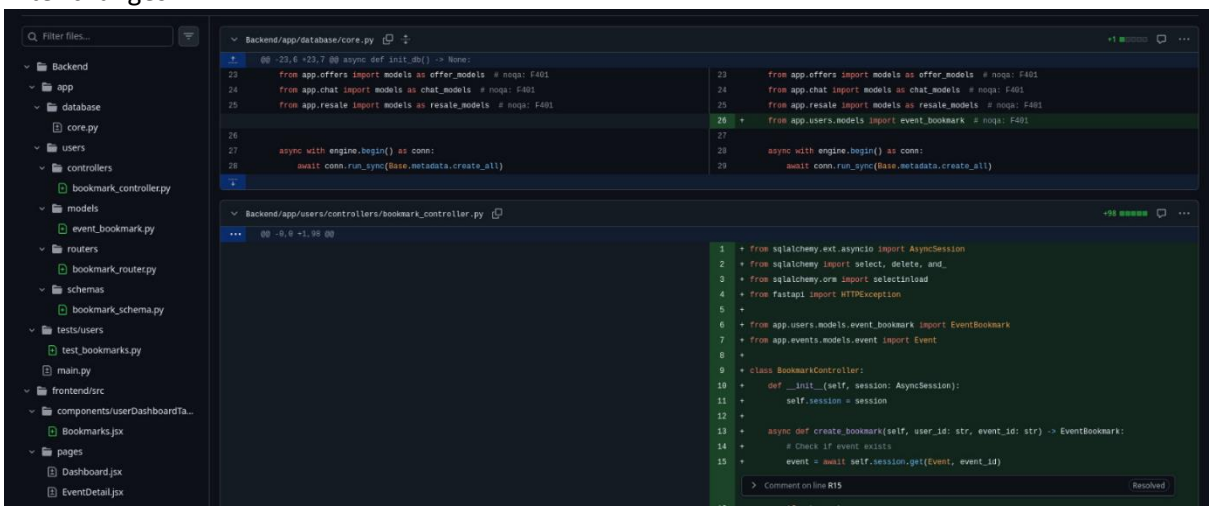
```
@@ -0,0 +1,26 @@
+ from __future__ import annotations
+
+ from typing import Optional
+
+ from sqlalchemy import select
+ from sqlalchemy.ext.asyncio import AsyncSession
+ from sqlalchemy.orm import selectinload
+
+ from app.tickets.models import Ticket
+
+
+ class TicketRepository:
+     """Persistence operations for tickets."""
+
+     def __init__(self, session: AsyncSession):
+         self._session = session
+
+     async def get_ticket_with_relations(self, ticket_id: str) -> Optional[Ticket]:
+         """Get a ticket with event and owner relationships eagerly loaded."""
+         stmt = (
+             select(Ticket)
+             .options(selectinload(Ticket.event), selectinload(Ticket.owner))
+             .where(Ticket.id == ticket_id)
+         )
+         result = await self._session.scalars(stmt)
+         return result.first()
```

Code Review #5

Feature: Bookmarking events
Code by: Bikrant
Reviewed by: Sahaj
Suggestion 1: refactoring into different layers



After changes:



5 Self-check on best practices for security

Passwords in the database are encrypted. Passwords are stored as hashed values in the database to ensure user credential security.

Protected assets:

name	Threats	Protection
User Data (Profiles, Credits)	Unauthorized access to personal details (email, name) or manipulation of reward points.	Access is secured via JWT Authentication (Access + Refresh tokens).Role-based access control enforces distinct permissions for Students, Organizers, and Admins.
Event & Ticket Data	Fraudulent ticket resale, scalping, or unauthorized modification of event details.	Ticket transfers are validated and processed securely via the internal marketplace logic.Only the "Organizer" role can modify their specific events, and "Admins" verify event approvals.
Media Files (Images/Videos)	Malicious file uploads (eg, scripts disguised as images) or storage overflow attacks.	Large binary objects are stored externally in AWS S3 to maintain DB performance.The system restricts uploads to standard web formats like PNG/JPEG.
Chat Messages	Privacy violations, harassment, or unauthorized eavesdropping on community discussions.	Chat rooms are restricted; only registered attendees of a specific event can access its corresponding chat.Real-time communication is handled via secure WebSockets, with moderation tools for reporting content.

Input data validation: We confirm that input data is validated to prevent injection attacks and ensure data integrity.

- **Search Bar Validation:**
  - **Constraint:** The search input is strictly validated to accept a maximum of **40 alphanumeric characters**.
  - **Implementation:** This is enforced on the frontend using React state validation before submission and on the backend using **FastAPI (Pydantic models)** to reject non-conforming queries before they reach the database SQL query execution.

6 Adherence to original Non-functional specs

1. Tools and Deployment The application must be developed, tested, and deployed using tools and servers approved by the Class CTO and agreed upon in Milestone 0. Application delivery must be from the chosen cloud server **Status:** DONE

2. Browser Compatibility The application must be optimized for standard desktop/laptop browsers, rendering correctly on the two latest versions of two major browsers **Status:** DONE

3. Mobile Rendering All or selected application functions are required to render well on mobile devices. (The project prioritizes desktop browsers but must use a responsive UI) **Status:** DONE

4. Data Storage Data must be stored in the database located on the team's deployment cloud server **Status:** DONE

5. Concurrency No more than 50 concurrent users shall be accessing the application at any time **Status:** DONE

6. Privacy The privacy of users must be protected, and all privacy policies must be appropriately communicated to the users **Status:** DONE

7. Language The language used must be English; no localization is needed **Status:** DONE

8. Usability The application must be very easy to use and intuitive **Status:** DONE

9. Architecture The application should follow established architecture patterns **Status:** DONE

10. Maintainability The application code and its repository must be easy to inspect and maintain **Status:** DONE

11. Analytics Google Analytics shall be used (this is optional for Fulda teams) **Status:** DONE

12. Email Constraint No email clients are allowed **Status:** DONE

13. Payment Constraint Payment functionality, if any, shall not be implemented or simulated in the UI **Status:** DONE

14. Security Basic best practices for site security (as covered in the class) must be applied for main data items **Status:** DONE

15. Media The application must be media-rich (images, video, etc.) using standard market formats **Status:** DONE

16. Processes Modern Software Engineering (SE) processes and practices must be used as specified in the class, including collaborative and continuous software development **Status:** DONE

17. Code Management Each team must use their own GitHub repository (set up by instructors) for code development, management, and required documentation such as formal milestones **Status:** DONE

18. Display Text The application UI must prominently display the following exact text on all pages at the top: "Fulda University of Applied Sciences Software Engineering Project, Summer 2025 For Demonstration Only **"Status:** DONE